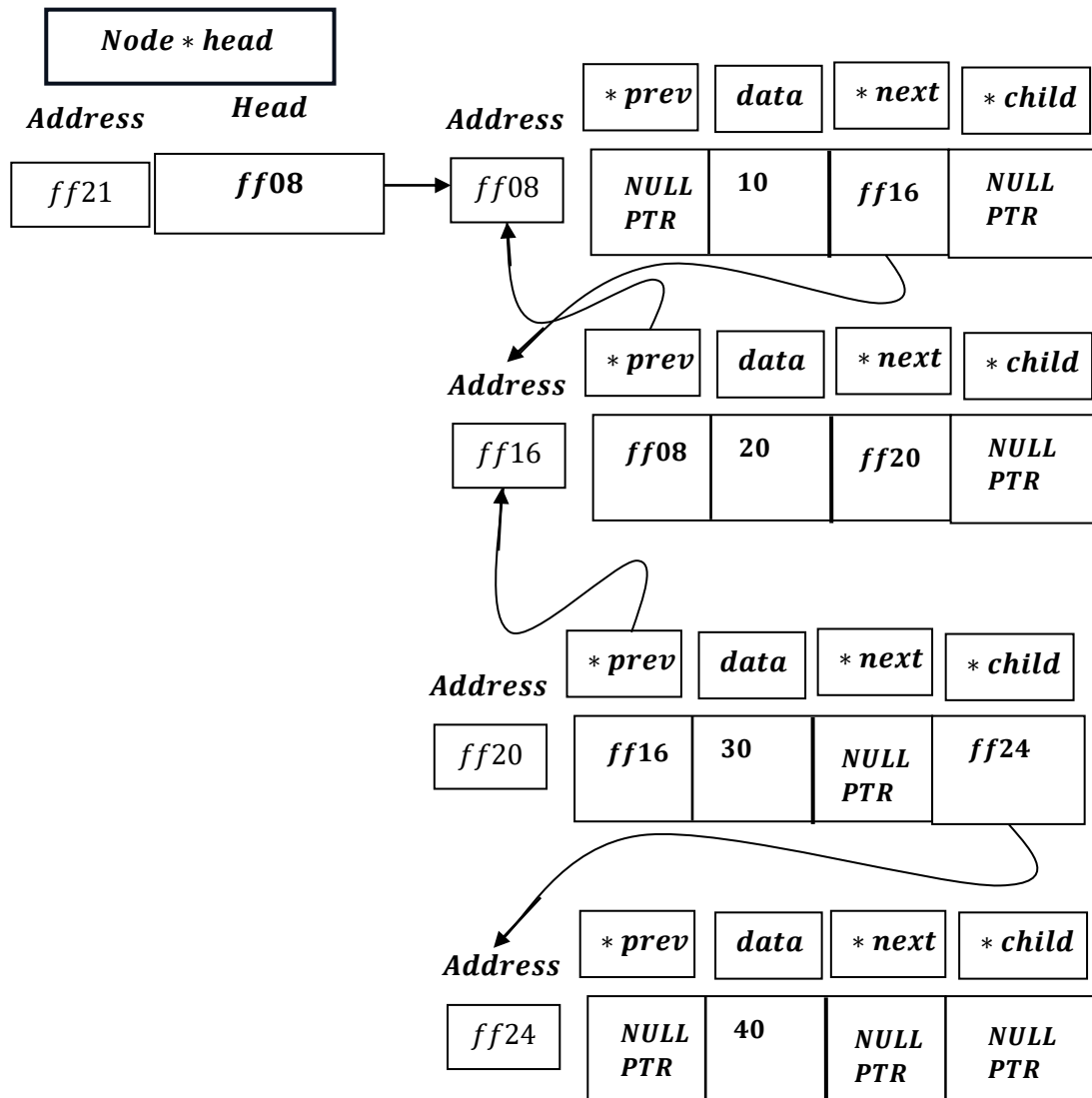
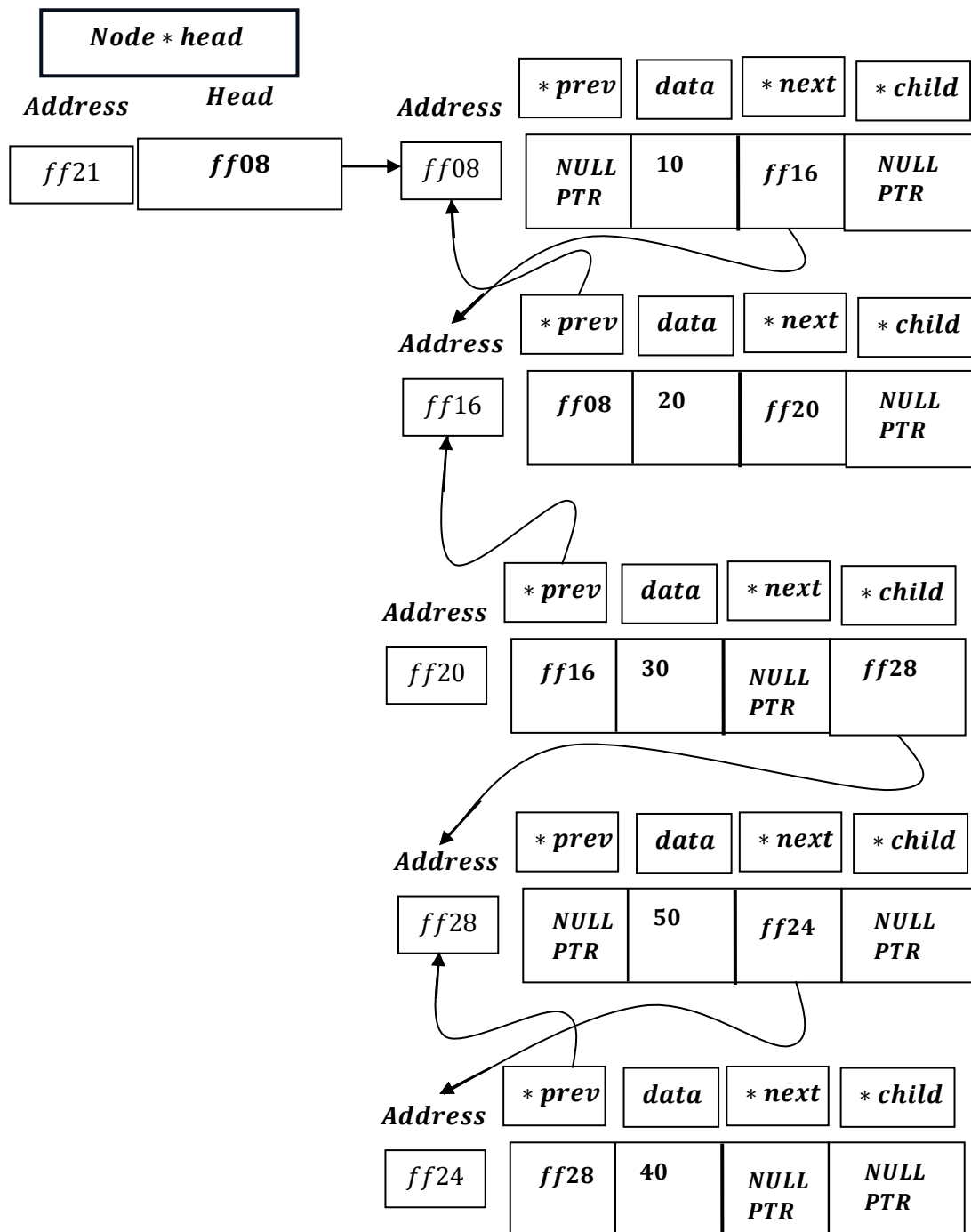


Multi – level Linked List – Different Versions – Doubly, Circular, Doubly Circular

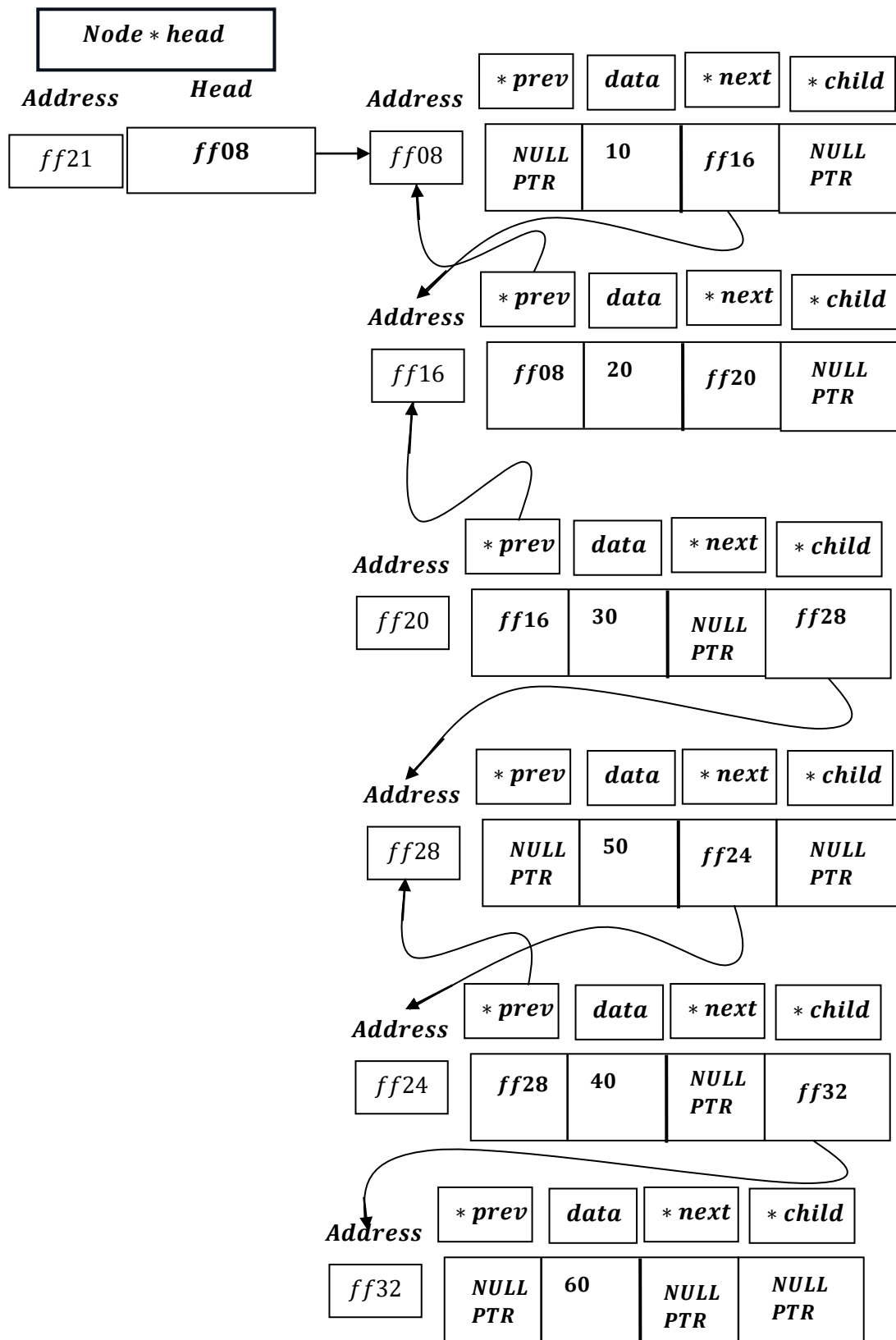
Multi – Level Doubly Linked List

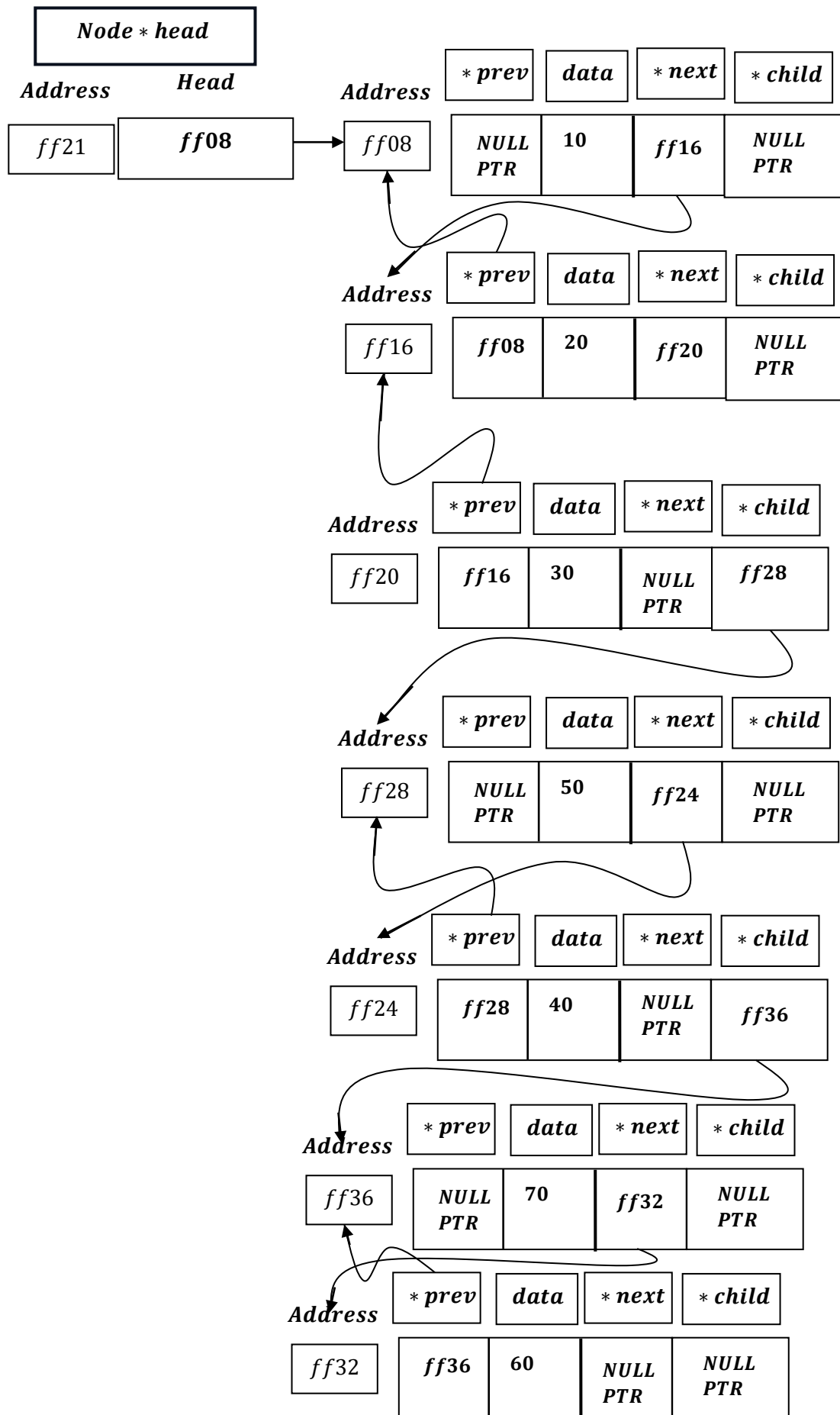


Now another child insertion:



Child's child insertion:





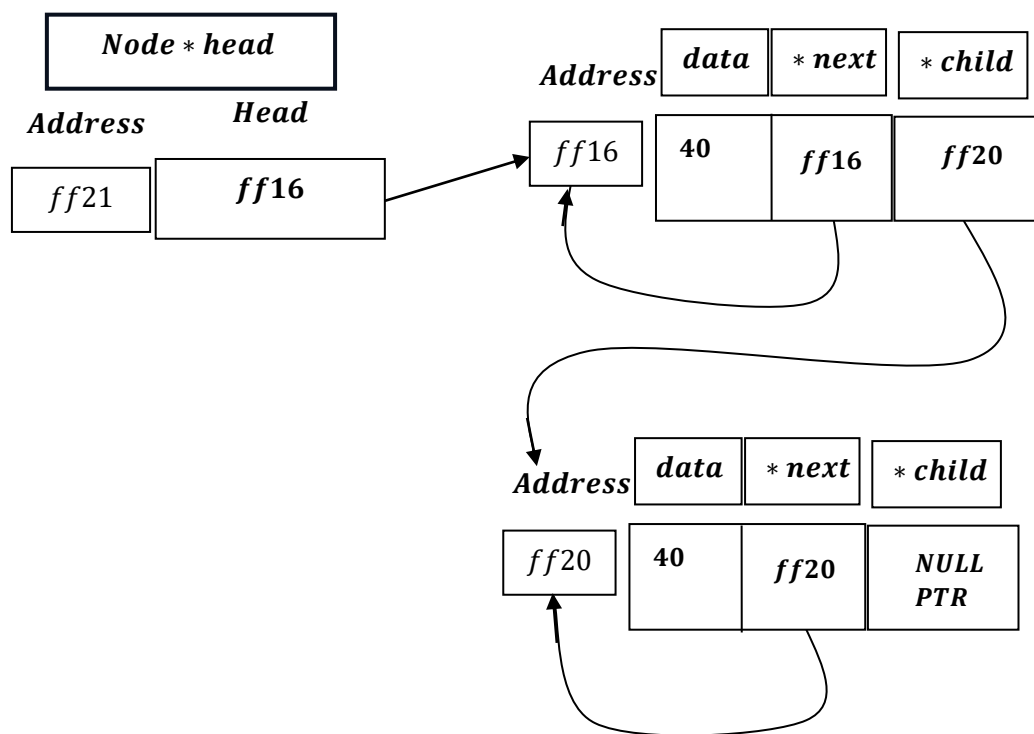
In Program

Doubly Linked List, a prev pointer has been added to the Node structure to allow two – way navigation on the same level. The insertion and deletion functions have been updated to correctly link and unlink these prev pointers.

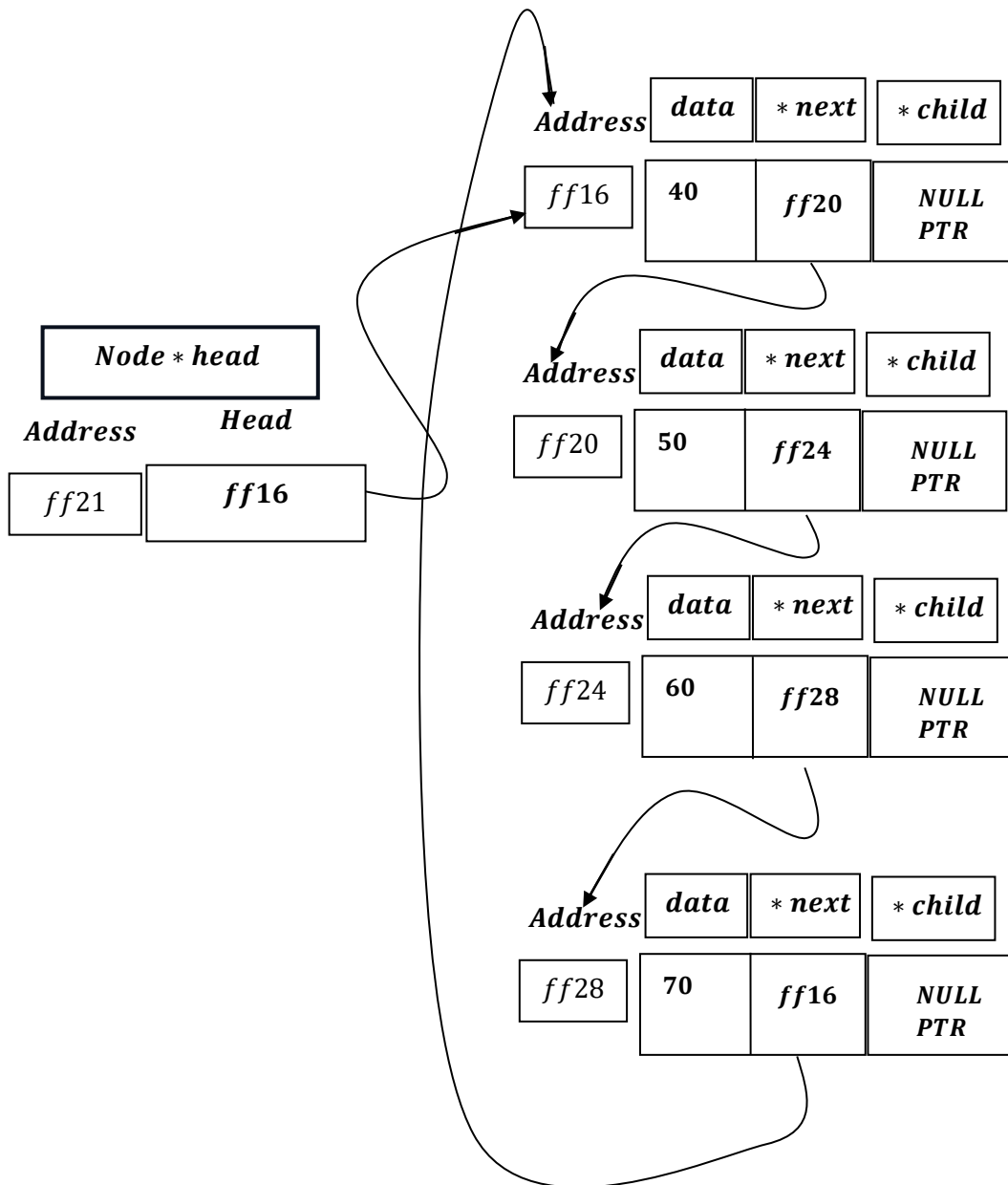
Multi – Level Circular Linked List

Main – List Circular

1. Single Node Insertion:

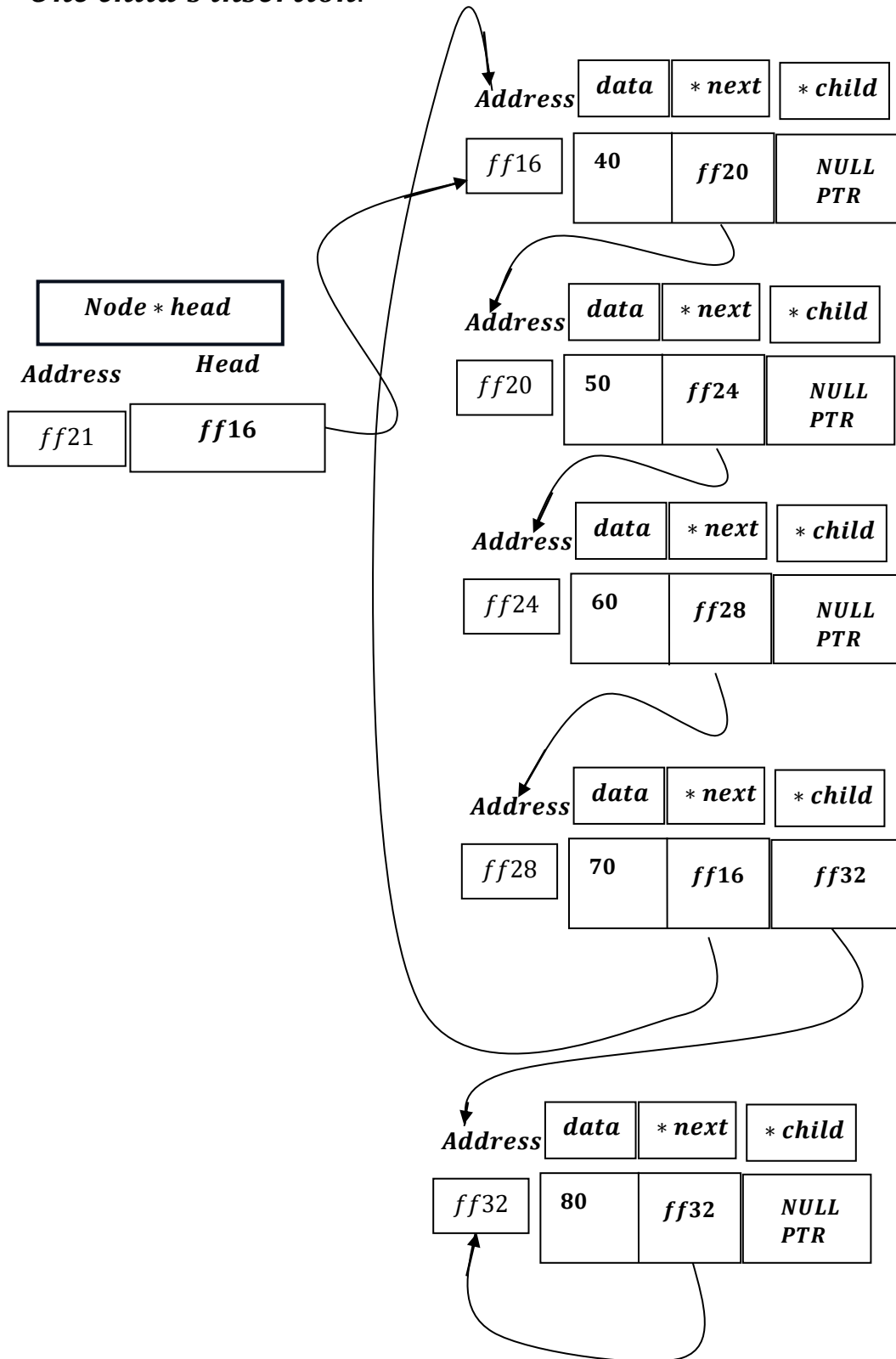


2. Multiple Node Insertion:

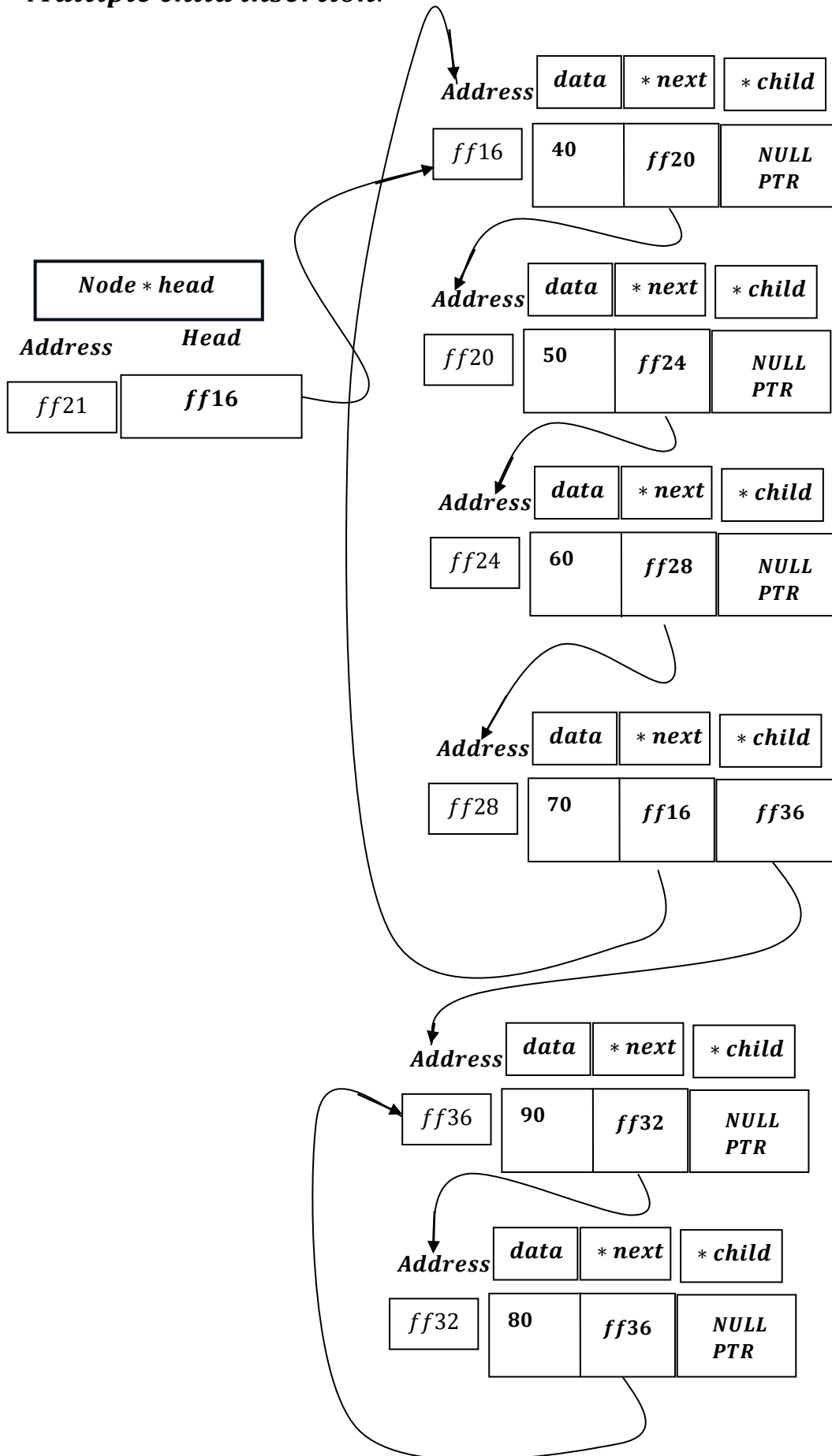


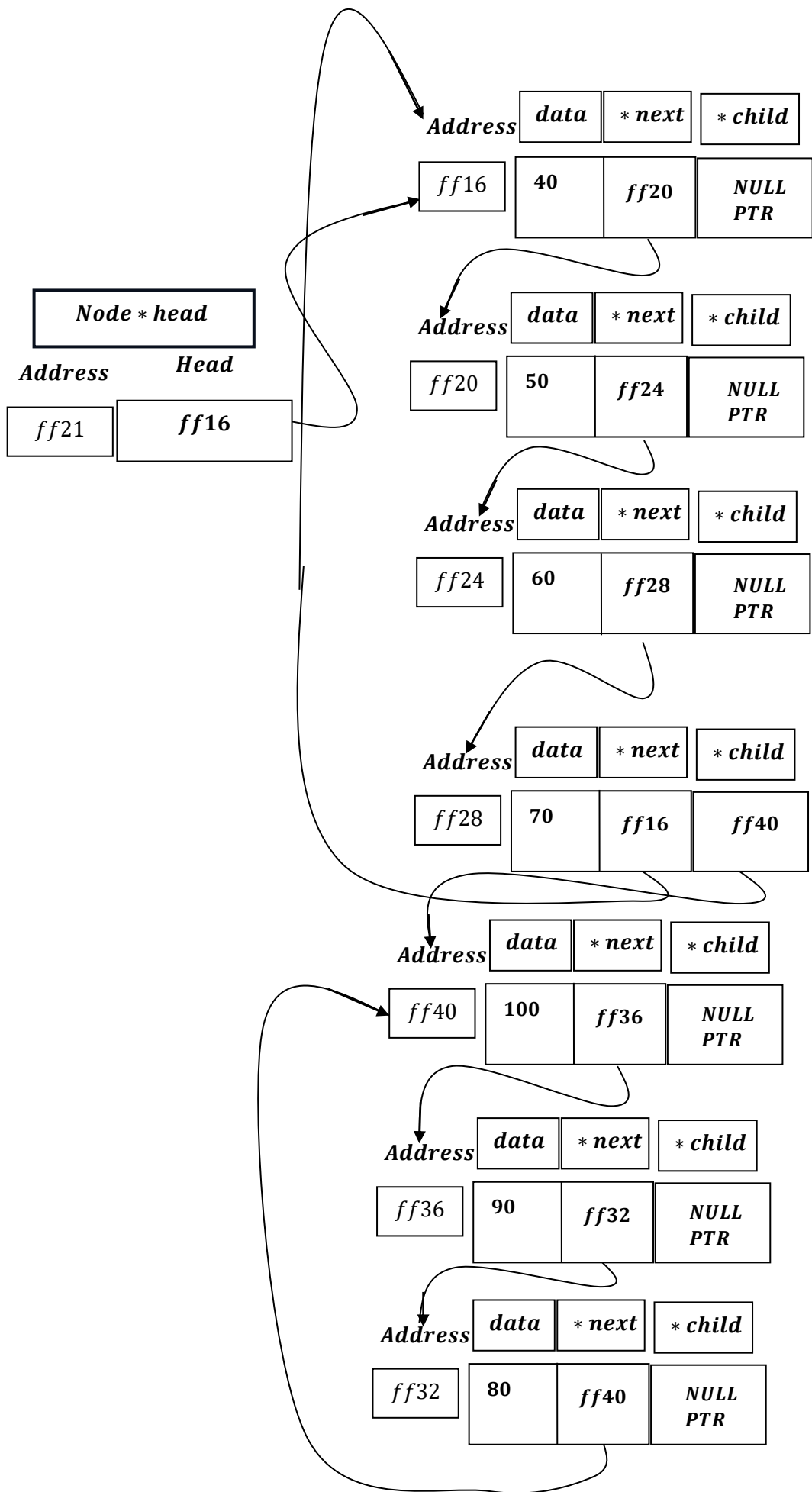
Child Node's Insertion

One child's insertion:



Multiple child insertion:





In Program

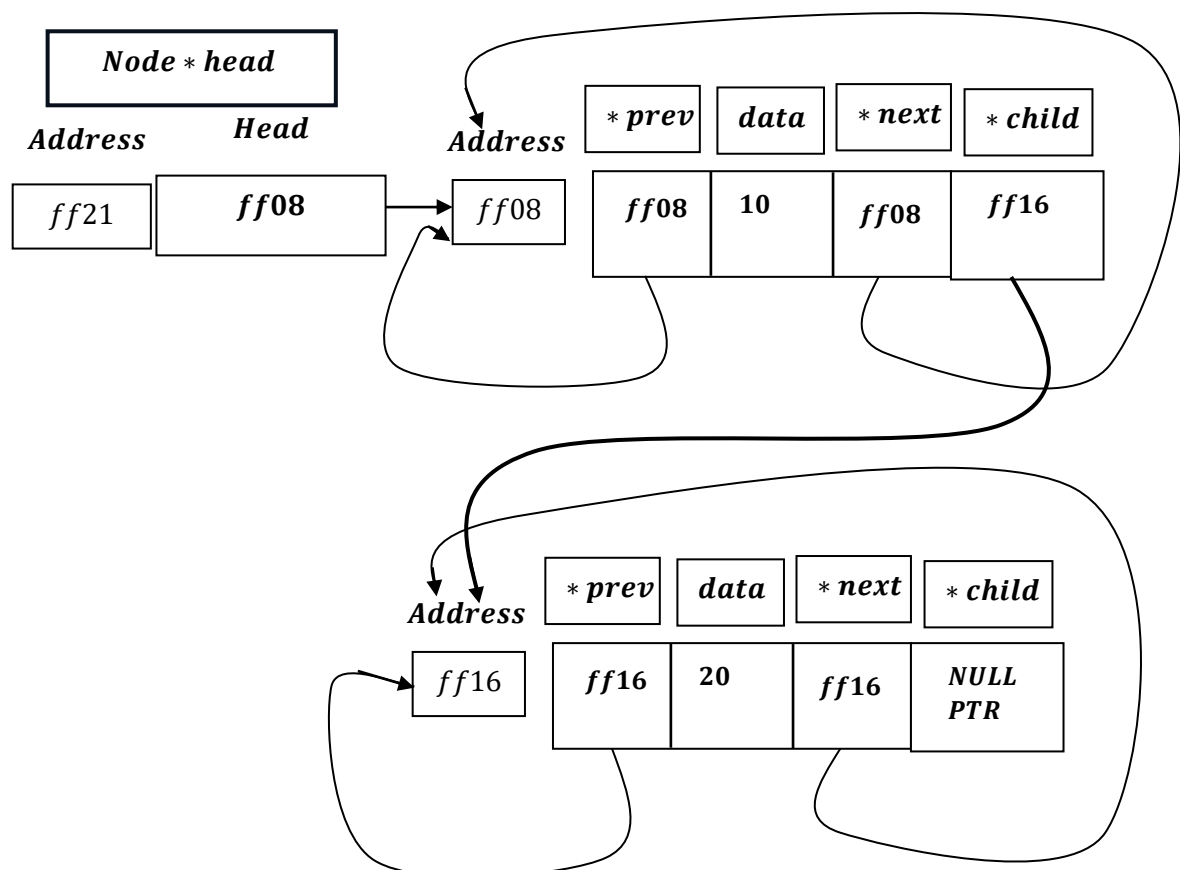
Circular Linked List, the most significant change is ensuring that the next pointer of the last node in any horizontal sequence points back to the first node (head) of that specific sequence, rather than nullptr.

Because of this circular nature, recursive traversals

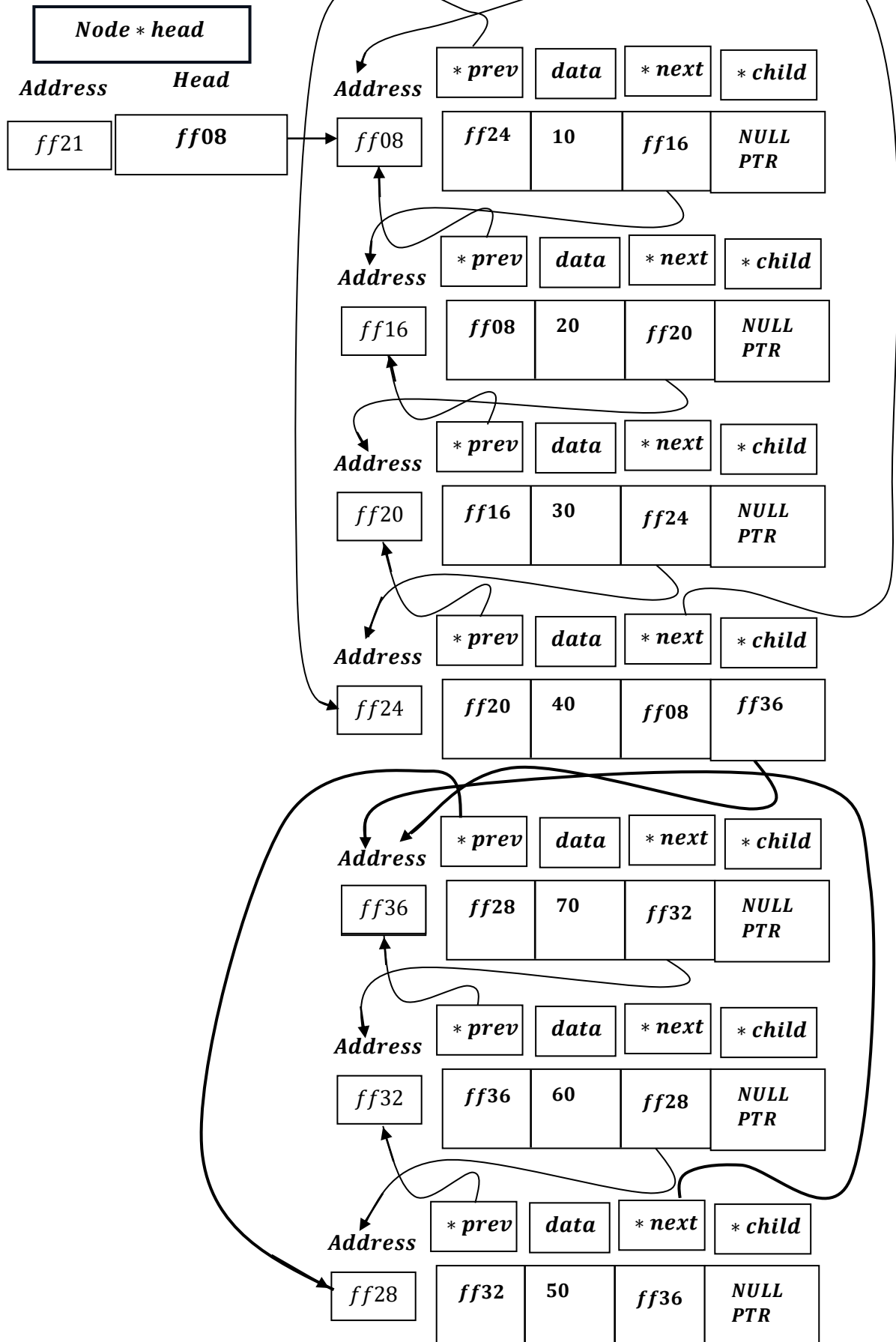
(like searching, printing, and destroying) must be carefully controlled using do – while loops or by breaking the circle to prevent infinite stack overflow errors.

Multi – Level Circular Doubly Linked List

One Single Node



Multi Node



In Program

- *Circular Linking: The next pointer of the last node at any level must wrap around to the first node (head) of that level, and the prev pointer of the first node must wrap around to the last node.*
- *Single Node Logic: When a list has only one element, its next and prev pointers must point to itself.*
- *Traversal Updates: Functions like `searchNode`, `printList`, and `deletion` traversals now use do – while loops
(or check for wrap – around) to avoid infinite loops.*
- *Destruction: The `destroyLevel` function breaks the circular link
(`tail → next = nullptr`) just before garbage collection
so it can recursively free nodes identically to a linear list
without stacking infinitely.*